



Trabalho prático: Resolver o problema de faturamento Cloud

Contexto do negócio:

A empresa **XYZ** aluga infraestrutura de TI para outras empresas. Atualmente, o sistema de faturamento (*Billing*) deles virou um monstro incontrolável.

O Problema Atual:

1. **O Acoplamento de Hierarquia:** Um cliente pode alugar uma única Máquina Virtual (VM), ou pode alugar um "Cluster" (que contém várias VMs), ou um "Data Center Virtual" (que contém vários Clusters). Hoje, o código tem dezenas de if (tipo == "Cluster") { for... } else if (tipo == "VM") { ... } espalhados por todo o sistema para conseguir calcular o preço total.
2. **A Explosão de Subclasses:** A empresa começou a vender serviços adicionais para as VMs. Eles tinham a classe VMBase. Aí criaram a VMComBackup. Depois a VMComFirewall. Agora eles têm classes bizarras como VMComBackupFirewallEAntiDDoS, gerando um acoplamento rígido e um código impossível de manter.

Missão:

Você foi contratado como Arquiteto de Software para refatorar o motor de precificação da XYZ. O sistema precisa ser capaz de:

1. Tratar itens individuais e agrupamentos complexos de forma idêntica (O cliente deve poder chamar `calcular_preco()` na raiz do projeto e o sistema se resolve).
2. Adicionar serviços extras (comportamentos e custos) a qualquer recurso de forma dinâmica, em tempo de execução, sem criar novas subclasses.

FASE 1: Modelagem Estrutural – Padrão Composite

Objetivo: Resolver a hierarquia de recursos.

Você deve criar a estrutura básica de recursos computacionais.

- **Componente Base:** Crie uma interface/classe abstrata `RecursoCloud` que tenha os métodos `get_descricao()` e `get_preco()`.
- **Folhas (Leafs):** Crie as classes `MaquinaVirtual` (Preço base: R\$ 100,00) e `BancoDeDados` (Preço base: R\$ 250,00). Ambas implementam `RecursoCloud`.

- **Compositor (Composite):** Crie a classe GrupoRecursos (representando Clusters ou Data Centers). Ela também implementa RecursoCloud, mas guarda uma lista de filhos. Seu `get_preco()` deve somar o preço de todos os filhos.

FASE 2: Extensão dinâmica – Padrão Decorator

Objetivo: Resolver a explosão de subclasses dos serviços adicionais.

Qualquer RecursoCloud (seja uma máquina avulsa ou um cluster inteiro) pode receber serviços adicionais.

- **Decorador Base:** Crie a classe abstrata `ServicoDecorator` que implementa `RecursoCloud` e recebe um `RecursoCloud` no seu construtor (o famoso *wrapper*).
- **Decoradores Concretos:**
 - `BackupAutomato`: Adiciona " + Backup" na descrição e soma R\$ 50,00 ao preço do recurso decorado.
 - `FirewallPremium`: Adiciona " + Firewall" na descrição e soma R\$ 80,00 ao preço.
 - `Suporte24x7`: Adiciona " + Suporte VIP" e soma **10%** ao preço total do recurso que ele está envolvendo (Atenção a esta regra de negócio!).

FASE 3: Integração e código Cliente

Objetivo: Provar que os padrões funcionam juntos para resolver o problema inicial.

Escreva o código principal (main) que simula a compra de um cliente corporativo. O script deve instanciar exatamente este cenário:

1. **Cluster Alpha:** Contém 2 Máquinas Virtuais simples e 1 Banco de Dados.
2. **Cluster Beta:** Contém 1 Máquina Virtual. Esta máquina específica deve ter **Backup** e **Firewall**.
3. **Data Center Principal:** Um grupo que engloba o *Cluster Alpha* e o *Cluster Beta*.
4. **A Regra de Ouro:** O cliente decidiu comprar o **Suporte24x7** para o *Data Center Principal* INTEIRO. (Ou seja, o decorador deve envolver o composite raiz).

No final, o sistema deve imprimir a descrição completa e recursiva da arquitetura e o preço total final.